

Coding-decoding

COMPLETE LEARNING GUIDE

What You'll Learn

Coding-decoding questions establish an artificial rule for transforming letters, numbers, or words, show one worked example of the rule in action, and then ask you to apply (or reverse) that same rule to a new input — the entire skill is pattern extraction from a single example, then mechanical, careful application. Common rule families include letter shifting (Caesar-style), numeric substitution, word-for-word substitution codes, coordinate/grid codes, and mixed patterns where odd and even positions shift differently — and the most common error is applying the discovered rule inconsistently rather than failing to find it. This sub-module works through each rule family with a fully solved example, including how to decode (reverse-apply) a rule, since decoding questions appear just as often as encoding ones and require running the pattern backward.

1. Letter Shifting (Caesar-Style) Codes

The most common coding rule shifts every letter forward or backward by a fixed number of positions in the alphabet — identify the shift from the given example word, then apply the identical shift to the new word.

- If CAT is coded as DBU, each letter shifted forward by 1 (C!D, A!B, T!U). Using the same rule, DOG codes as: D!E, O!P, G!H !' EPH.

2. Numeric Substitution Codes

Numeric codes typically assign each letter its alphabetical position (A=1...Z=26) and then apply a further arithmetic operation (doubling, adding a constant) — extract both steps from the example before applying them to the new word.

- If A=1, B=2, ..., Z=26, and each value is then doubled: CAT !' C=3!6, A=1!2, T=20!40. Coded as 6-2-40.

3. Word-for-Word Substitution Codes

Some questions code entire sentences, assigning each word an arbitrary code word — the technique is to find words common to two or more coded sentences and match them against the words common to the corresponding plain sentences.

- 'red is blue' is coded 'pa ta la'. 'blue is sky' is coded 'la ta ka'. The word common to both plain sentences is 'is'; the code word common to both coded sentences is 'ta'. So 'ta' means 'is'.

4. Matrix (Coordinate/Grid) Codes

Grid-based codes place letters into a numbered row-column matrix, and each letter is represented by its (row, column) coordinate pair — once the grid layout is given or inferred from an example, decoding is a direct lookup.

5. Decoding: Running the Rule Backward

Decoding questions give you the coded word and the rule (or an example pair) and ask for the original word — apply the inverse operation of whatever the encoding did (shift backward instead of forward, halve instead of double).

- If EDUCATION is coded as FEVDBUJPO (each letter shifted forward by 1), decode GSJFOE by shifting each letter BACKWARD by 1: G!'F, S!'R, J!'I, F!'E, O!'N, E!'D !' FRIEND.

6. Symbol-for-Letter Codes

When a question provides a fixed key mapping symbols to letters (e.g., @ = A, # = B, % = C), decoding or encoding is a direct substitution using that key — the only real risk is a careless lookup error under time pressure, so double-check each symbol against the key rather than relying on memory after the first few.

7. Mixed and Position-Dependent Patterns

The hardest coding questions apply different rules to different positions in a word (e.g., letters in odd positions shift +1, letters in even positions shift " 1) — identify this by checking whether a single uniform shift explains the given example; if it doesn't fit cleanly, test position-dependent rules next.

- If a single shift doesn't explain a given example word consistently across all letters, check whether odd-position letters and even-position letters are shifting by different amounts — a common design for higher-difficulty coding questions.

Common Pitfalls

- Applying an inconsistent shift value across the letters of a word instead of double-checking the SAME shift explains every letter in the given example first.
- Forgetting to reverse the operation (shift backward, halve instead of double) when a question asks you to decode rather than encode.
- In word-substitution codes, matching the wrong pair of words between two coded sentences instead of finding the genuinely common word.
- Assuming a single uniform rule when the pattern is actually position-dependent (different rule for odd vs even letter positions).

- Misreading the alphabet position of a letter near the end (like X, Y, Z) and wrapping around incorrectly without checking whether the rule intends wrap-around at all.
- Not verifying the discovered rule against the FULL given example word before applying it to the new word, missing a rule that only partially fits.

Key Takeaways

- Extract the coding rule strictly from the given example — verify it explains every letter/element before applying it elsewhere.
- Decoding is simply running the encoding rule in reverse (shift backward instead of forward, halve instead of double).
- In sentence/word-substitution codes, match the word common to two plain sentences against the code word common to their coded versions.
- If a uniform shift doesn't fit the example cleanly, test position-dependent (odd/even) rules next.
- Numeric letter codes usually combine an alphabet-position step (A=1...Z=26) with a further arithmetic operation — identify both separately.
- Double-check symbol/grid lookups against the given key each time rather than relying on memory partway through.

Learn More

IndiaBix — Coding-Decoding Questions

<https://www.indiabix.com/logical-reasoning/coding-decoding/>

GeeksforGeeks — Logical Reasoning Questions and Answers

<https://www.geeksforgeeks.org/logical-reasoning-questions-and-answers/>